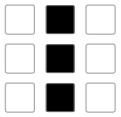
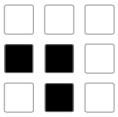
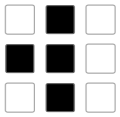
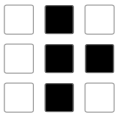
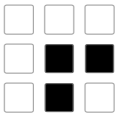
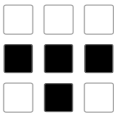
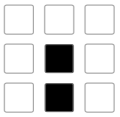
1. Phase 1 of the report

Since the maze is not drawn with uniform intervals, we did not choose an approach that maps each node to absolute (x, y) coordinates.

The IR sensors are used to recognize the type of current position(Node). Each Node falls into one of the following types: straight, left, right, branch, or wall.

straight	left		right		branch	wall
	(a)	(b)	(a)	(b)		
						

Rotation is implemented based on IR sensor recognition rather than controlling the motor using time units.
Note that corners and branches can be classified into six types as follows:

a. Node_Branch & Dir_L
b. Node_Branch & Dir_R
c. Node_Left & Dir_L
d. Node_Left & Dir_S
e. Node_Right & Dir_R
f. Node_Right & Dir_S

In Phase_One_Two(), the program enters Escape_And_Memorize_Maze().
Note that we followed a left-priority rule: left (Dir_L) > straight (Dir_S) > right (Dir_R). Hence, Dir_L is favored in (b) of left node, Dir_S is favored in (a) of right node, and Dir_L is favored in branch node.
Depending on the type of the node, the robot moves as follows:
- Node_Branch : Turn left, then push the node type Node_Branch and direction Dir_L onto node_stack and dir_stack, respectively.

- Node_Left : Select direction based on priority, push the node type Node_Left and direction Dir_L onto node_stack and dir_stack, respectively.
- Node_Right : Select direction based on priority, push the node type Node_Right and direction Dir_S onto node_stack and dir_stack, respectively.
- Node_Straight : Move straight until another node type is detected.
- Node_Wall : Count the number of backtracks and refer to the previously stored stack values. Depending on the stored direction, the backtrack is handled by either Process_Backtrack_Branch(), Process_Backtrack_Left(), or Process_Backtrack_Right(). Within each function, the backtrack behavior is further determined by the node type stored in the node_stack.
- Node_End : Returns 1 to indicate the maze has been successfully exited.

In the Escape_And_Memorize_Maze(), the first step is to check whether backtrack_count is greater than 0. If it is, a backtrack operation is performed by referring to the top values of the stacks.
According to the six types of corners and branches mentioned earlier, the state transition occurs as follows: type a -> type b, type c -> type d, and type f -> type e. If a backtrack is required even in types b, d, or e, the robot reverts to the before entering the respective corner or branch, and a pop operation is performed on both stacks.

Through the process of pathfinding and backtracking, when the robot eventually reaches the Node_End, each stack retains the path information that avoids dead ends and unnecessary backtracking.

In Phase_Three(), based on the information previously stored in node_stack and dir_stack, the robot makes optimal decisions at each corner or branch, allowing it to escape the maze more quickly than in Phase_One_Two().

2. Phase 2 of the report

Predict 1: It is expected that the robot will move reliably, enabling it to follow the planned algorithm to escape the maze and accurately memorize the path.

Problem 1: Due to motor output errors, the robot may move farther or shorter than expected, or rotate more or less than intended, leading to incorrect recognition of the corner or branch type by the IR sensor.

Predict 2: In Phase_Three(), the robot is expected to escape the maze more quickly based on the path memorized during Phase_One_Two(), specifically the values stored in node_stack and dir_stack (e.g., Node_Branch → Node_Right → ... and Dir_L → Dir_S → Dir_R → ...).

Problem 2: Due to motor output errors and incorrectly mapped IR sensor readings, the robot may mistakenly interpret a straight path as a corner or branch and perform unintended movements. This leads to corrupted memory and prevents the robot from escaping as expected.

Predict 3: Since the mapping of Node_End (0b 0111 1110) does not appear as a branch during the actual maze navigation, it was assumed that the robot would be able to exit the maze successfully.

Problem 3: Due to motor output errors, the robot occasionally misrecognizes a branch as Node_End or failed to recognize Node_End at all.

Predict 4: It was assumed that the output would remain relatively stable until the battery was fully discharged.

Problem 4: As the battery level decreased, the output dropped significantly, causing the robot to behave unexpectedly.